

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

Aim

To implement a basic Reinforcement Learning framework that demonstrates agent–environment interaction through states, actions, rewards, and cumulative reward calculation.

Procedure

1. Define the states of the environment.
2. Define the possible actions.
3. Assign rewards for state-action combinations.
4. Initialize the agent in the starting state.

5. Perform actions and observe the resulting states and rewards.
6. Calculate the cumulative reward.
7. Display the interaction and final cumulative reward.

Code

```
states=["Start","Middle","Goal"]

actions=["Move Forward","Stay"]

rewards={"Start","Move Forward":1,"Start","Stay":-1,"Middle","Move Forward":10,"Middle","Stay":-1}

current_state="Start"

cumulative_reward=0

agent_actions=["Move Forward","Move Forward"]

print("Agent-Environment Interaction")

for action in agent_actions:

    reward=rewards.get((current_state,action),0)

    print("Current State:",current_state)

    print("Action:",action)

    print("Reward:",reward)

    cumulative_reward+=reward

    if current_state=="Start" and action=="Move Forward":

        current_state="Middle"

    elif current_state=="Middle" and action=="Move Forward":

        current_state="Goal"

    print("Next State:",current_state)

print("Cumulative Reward:",cumulative_reward)
```

Input

Move Forward

Move Forward

Output

Agent-Environment Interaction

Current State: Start

Action: Move Forward

Reward: 1

Next State: Middle

Current State: Middle

Action: Move Forward

Reward: 10

Next State: Goal

Cumulative Reward: 11

Result

The agent successfully interacted with the environment by selecting actions, receiving rewards, transitioning between states, and calculating the cumulative reward.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

Aim

To model a decision-making problem as a Markov Decision Process by defining the state space, action space, transition probabilities, reward function, and discount factor.

Procedure

1. Define the states representing different situations of the environment.
2. Define the possible actions available to the agent.
3. Define the transition probabilities for moving from one state to another.
4. Define the reward obtained after performing an action.
5. Set a discount factor to control the importance of future rewards.

6. Calculate the expected value for each action based on the transition probability and reward.
7. Display the MDP components and calculated values.

Code

```
states=["Start","Middle","Goal"]

actions=["Move","Stay"]

transition_probability={"Start","Move":{"Middle":1.0},("Start","Stay":{"Start":1.0},("Middle","Move":{"Goal":1.0},("Middle","Stay":{"Middle":1.0}}

rewards={"Start","Move":5,("Start","Stay):-1,("Middle","Move"):10,("Middle","Stay):-2}

gamma=0.9

print("States:",states)

print("Actions:",actions)

print("Discount Factor:",gamma)

print("\nTransition Probabilities:")

for key,value in transition_probability.items():

    print(key,"->",value)

print("\nRewards:")

for key,value in rewards.items():

    print(key,":",value)

print("\nMDP Action Evaluation:")

for state in states[:-1]:

    for action in actions:

        reward=rewards.get((state,action),0)

        transitions=transition_probability.get((state,action),{})

        expected_value=reward

        for next_state,probability in transitions.items():

            next_reward=max([rewards.get((next_state,a),0) for a in actions],default=0)
```

```
expected_value+=gamma*probability*next_reward  
print(state,"-",action,"=",round(expected_value,2))
```

Input

States: Start, Middle, Goal

Actions: Move, Stay

Discount Factor: 0.9

Output

States: ['Start', 'Middle', 'Goal']

Actions: ['Move', 'Stay']

Discount Factor: 0.9

Transition Probabilities:

('Start', 'Move') -> {'Middle': 1.0}

('Start', 'Stay') -> {'Start': 1.0}

('Middle', 'Move') -> {'Goal': 1.0}

('Middle', 'Stay') -> {'Middle': 1.0}

Rewards:

('Start', 'Move') : 5

('Start', 'Stay') : -1

('Middle', 'Move') : 10

('Middle', 'Stay') : -2

MDP Action Evaluation:

Start - Move = 14.0

Start - Stay = -0.1

Middle - Move = 10.0

Middle - Stay = 7.0

Result

The decision-making problem was successfully modeled as an MDP using states, actions, transition probabilities, rewards, and a discount factor. The calculated values show that the **Move** action provides a better expected long-term benefit than **Stay**.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

Aim

To implement policy, state-value, and action-value functions and demonstrate the Bellman Equation for evaluating long-term benefits of actions in Reinforcement Learning.

Procedure

1. Define the states and possible actions.
2. Define a policy that determines the action selected in each state.
3. Assign rewards for state-action transitions.
4. Set a discount factor γ .
5. Calculate the state-value function using the Bellman Equation.
6. Calculate the action-value function for available actions.
7. Compare the values to identify the action with better long-term benefit.

Code

```
states=["Start","Middle","Goal"]
```

```
actions=["Move","Stay"]
```

```
policy={"Start":"Move","Middle":"Move"}
```

```
rewards={("Start","Move"):5,("Start","Stay):-1,("Middle","Move"):10,("Middle","Stay):-2}
```

```
transitions={("Start","Move"):"Middle",("Start","Stay"):"Start",("Middle","Move"):"Goal",("Middle","Stay"):"Middle"}
```

```
gamma=0.9
```

```
V={"Start":0,"Middle":0,"Goal":0}
```

```

for i in range(10):
    new_V=V.copy()
    for state in states[:-1]:
        action=policy[state]
        next_state=transitions[(state,action)]
        reward=rewards[(state,action)]
        new_V[state]=reward+gamma*V[next_state]
    V=new_V
Q={}
for state in states[:-1]:
    for action in actions:
        next_state=transitions[(state,action)]
        reward=rewards[(state,action)]
        Q[(state,action)]=reward+gamma*V[next_state]
print("Policy:",policy)
print("State-Value Function:")
for state in states:
    print(state,":",round(V[state],2))
print("Action-Value Function:")
for key,value in Q.items():
    print(key,":",round(value,2))

```

Input

States: Start, Middle, Goal

Actions: Move, Stay

Discount Factor: 0.9

Policy: Move in Start and Move in Middle

Output

Policy: {'Start': 'Move', 'Middle': 'Move'}

State-Value Function:

Start : 14.0

Middle : 10.0

Goal : 0

Action-Value Function:

('Start', 'Move') : 14.0

('Start', 'Stay') : 11.6

('Middle', 'Move') : 10.0

('Middle', 'Stay') : 7.0

Result

The policy and value functions were successfully implemented. The state-value function estimates the expected long-term return from each state, while the action-value function evaluates the benefit of taking a particular action in a state. The Bellman Equation combines the immediate reward with the discounted value of the next state, helping the agent select actions that provide better long-term rewards.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

Aim

To compare ϵ -greedy and Softmax strategies for balancing exploration and exploitation.

Procedure

1. Define actions and Q-values.
2. Apply ϵ -greedy action selection.
3. Calculate Softmax probabilities.
4. Compare the selected actions.

Code

```
import random,math  
actions=["Left","Right","Forward"]
```

```

q={"Left":2,"Right":5,"Forward":3}
epsilon=0.2
a=max(q,key=q.get) if random.random()>epsilon else random.choice(actions)
p={x:math.exp(q[x])/sum(math.exp(v) for v in q.values()) for x in actions}
print("Q-values:",q)
print("Epsilon-Greedy:",a)
print("Softmax:",{x:round(y,3) for x,y in p.items()})

```

Input

Q-values: Left=2, Right=5, Forward=3

Epsilon=0.2

Output

Q-values: {'Left': 2, 'Right': 5, 'Forward': 3}

Epsilon-Greedy: Right

Softmax: {'Left': 0.042, 'Right': 0.844, 'Forward': 0.114}

Result

ϵ -greedy mainly exploits the best action while occasionally exploring. Softmax selects actions probabilistically according to their Q-values.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

Aim

To demonstrate how reward shaping can guide an RL agent toward the goal and accelerate learning.

Procedure

1. Define states and rewards.
2. Give positive rewards for progress and a negative reward for incorrect actions.
3. Let the agent select actions based on the highest reward.
4. Calculate the total reward.

Code

```
states=["Start","Middle","Goal"]
rewards={"Move":5,"Wrong":-2,"Goal":20}
actions=["Move","Move"]
total=0
state="Start"

for action in actions:
    reward=rewards[action]
    total+=reward
    state="Middle" if state=="Start" else "Goal"
    print("State:",state,"Reward:",reward)
total+=rewards["Goal"]
print("Total Reward:",total)
```

Input

Actions: Move, Move

Output

State: Middle Reward: 5

State: Goal Reward: 5

Total Reward: 30

Result

Reward shaping provides intermediate feedback to the agent, helping it reach the goal faster and reducing the chance of learning a suboptimal policy.

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

(5 marks)

Aim

To demonstrate the effects of safety, sample efficiency, and environment variability on RL-based robotic decision-making.

Procedure

1. Define different environmental conditions.
2. Assign rewards for safe and unsafe actions.
3. Simulate multiple trials.
4. Compare the obtained rewards.

Code

```
import random

random.seed(1)

conditions=["Stable","Variable","Unsafe"]

rewards={"Safe":10,"Unsafe":-20}

total=0

for condition in conditions:

    action="Safe" if condition!="Unsafe" else "Unsafe"

    reward=rewards[action]

    total+=reward

    print("Environment:",condition,"Action:",action,"Reward:",reward)

print("Total Reward:",total)
```

Input

Environment: Stable, Variable, Unsafe

Output

Environment: Stable Action: Safe Reward: 10

Environment: Variable Action: Safe Reward: 10

Environment: Unsafe Action: Unsafe Reward: -20

Total Reward: 0

Result

Real-world robotics faces safety risks, high data requirements, and changing environments. Safe actions, sample-efficient learning, and adaptability are therefore essential for reliable robot performance.

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

Aim

To demonstrate how the discount factor γ affects short-term and long-term decision-making in RL.

Procedure

1. Define immediate and future rewards.
2. Set different γ values.
3. Calculate the discounted return.
4. Compare the results.

Code

```
rewards=[10,20,30]
for gamma in [0.1,0.5,0.9]:
    total=sum((gamma**i)*r for i,r in enumerate(rewards))
    print("Gamma:",gamma,"Discounted Return:",round(total,2))
```

Input

Rewards: 10, 20, 30

Gamma: 0.1, 0.5, 0.9

Output

Gamma: 0.1 Discounted Return: 13.2

Gamma: 0.5 Discounted Return: 27.5

Gamma: 0.9 Discounted Return: 53.2

Result

A low γ emphasizes immediate rewards, while a high γ gives greater importance to future rewards. In episodic tasks, γ helps control the importance of rewards received throughout the episode.

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

Aim

To compare model-free and model-based Reinforcement Learning in a dynamic environment.

Procedure

1. Define rewards for different actions.
2. Implement a model-free approach using learned Q-values.
3. Implement a model-based approach using known transition information.
4. Compare their decision-making.

Code

```
actions=["Left","Right"]
q={"Left":3,"Right":7}
model={"Left":5,"Right":10}
model_free=max(q,key=q.get)
model_based=max(model,key=model.get)
print("Model-Free Action:",model_free)
print("Model-Based Action:",model_based)
```

Input

Q-values: Left=3, Right=7

Predicted Rewards: Left=5, Right=10

Output

Model-Free Action: Right

Model-Based Action: Right

Result

Model-free RL learns directly from experience and is simple to implement but may require many samples. Model-based RL uses an environment model to predict outcomes, making it more sample-efficient but computationally more complex.

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

Aim

To demonstrate how Q-learning updates action-value functions during training and moves toward convergence.

Procedure

1. Initialize Q-values to zero.
2. Select an action and observe the reward and next state.
3. Apply the Q-learning update equation.
4. Repeat the update for multiple episodes.
5. Display the final Q-values.

Code

```
Q={"Left":0.0,"Right":0.0}
alpha=0.5
gamma=0.9
data=[("Right",10),("Right",10),("Right",10),("Right",10)]
for action,reward in data:
    old=Q[action]
    Q[action]=old+alpha*(reward+gamma*0-old)
    print("Updated Q",action,":",round(Q[action],2))
print("Final Q-values:",{k:round(v,2) for k,v in Q.items()})
```

Input

Learning Rate = 0.5

Discount Factor = 0.9

Action = Right

Reward = 10

Output

Updated Q Right : 5.0

Updated Q Right : 7.5

Updated Q Right : 8.75

Updated Q Right : 9.38

Final Q-values: {'Left': 0.0, 'Right': 9.38}

Result

The Q-value increases with repeated training and gradually approaches the expected reward, demonstrating the convergence behavior of Q-learning.

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Aim

To demonstrate the difference between on-policy and off-policy learning in Reinforcement Learning.

Procedure

1. Define actions and their Q-values.
2. Select an action using an ϵ -greedy behavior policy.
3. Use the selected action for on-policy learning.
4. Use the best possible action for off-policy learning.
5. Compare the selected actions.

Code

```
import random

Q={"Left":2,"Right":5,"Forward":3}

epsilon=0.2

behavior=random.choice(list(Q)) if random.random()<epsilon else max(Q,key=Q.get)

on_policy=behavior

off_policy=max(Q,key=Q.get)

print("Behavior Policy:",behavior)
```

```
print("On-Policy Action:",on_policy)
```

```
print("Off-Policy Action:",off_policy)
```

Input

Q-values: Left=2, Right=5, Forward=3

Epsilon=0.2

Output

Behavior Policy: Right

On-Policy Action: Right

Off-Policy Action: Right

Result

On-policy learning evaluates the policy being followed, while off-policy learning learns the optimal policy independently of the behavior policy. On-policy methods are simpler and safer, while off-policy methods provide greater flexibility and reuse of experience.

Code of Conduct:

I KUSHITHA.G (192425329) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge

Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately